SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

This System Analysis Documentation (SAD) outlines the technical blueprint for RigRefine AI, the autonomous data engineering engine designed to eliminate manual data cleaning in heavy industries.

SYSTEM ANALYSIS DOCUMENTATION (SAD): RigRefine AI

1. Introduction

RigRefine AI is a strategic response to the "Data Janitor" bottleneck in the Oil & Gas (O&G) sector. While traditional systems require manual scripting to clean messy rig logs, RigRefine AI introduces a Stateful Decision Engine that automates schema inference, code generation, and error recovery.

2. Background

Currently, O&G data exists in "silos" of varying quality—ranging from legacy 1990s sensor logs to modern IoT streams.

- ***Previous Version: Manual Excel-based sorting or rigid, hard-coded Python scripts that break whenever a sensor format changes.***
- ***New Capabilities: Autonomous "Self-Healing" logic and AI-driven schema mapping via GLM 5.1.***

3. Target Stakeholders

Stakeholder	*Role*	*Expectations*
Junior Engineer	*Data Uploader / Curator*	*Rapid transformation of raw logs into usable CSVs/Dashboards.*
QA Specialist	*Compliance Verifier*	*Transparent audit trails and validation of AI "decisions."*
Plant Manager	*Decision Maker*	*High-level insights without waiting months for data prep.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Dev Team	*System Architects*	*Modular API design and secure sandbox for code execution.*

4. System Architecture & Design

4.1 High-Level Architecture

- *Type: Web-based AI Agentic Platform.*
- *Architecture: Client-Server with an Autonomous Execution Sandbox.*

RigRefine AI uses a Next.js frontend to handle file ingestion. The FastAPI backend serves as the orchestrator, sending data samples to GLM 5.1 for reasoning. The generated Python code is executed in a secure, isolated container (Sandbox) to prevent system-level interference.

4.2 LLM as a Service Layer (Dependency Diagram)

1. *Prompt Construction: The system takes the first 20 rows of a CSV + User instructions (e.g., "Normalize Pressure") and wraps them in a System Role Prompt.*
2. *Context Window: Handled by sending schema definitions and sample data (max 2k tokens) to ensure the model understands the physical units (PSI, Bar, Celsius).*
3. *Response Parsing: The system extracts the Python block from the GLM 5.1 response using regex and passes it to the Executor component.*
4. *Token Enforcement: Inputs are truncated if headers exceed 100 columns; row data is sampled rather than sent in bulk to keep costs manageable.*

4.3 Sequence Diagram: The "Refine" Flow

1. *User uploads messy CSV.*
2. *Backend extracts headers and metadata.*
3. *GLM 5.1 analyzes metadata and returns a "Cleaning Plan" + Python Script.*
4. *Sandbox runs the script.*
5. *QA Logic checks for success. If FAIL, error logs are sent back to GLM 5.1 for a "Self-Healing" fix.*
6. *Frontend displays the "Before/After" results.*

4.4 Technological Stack

- *Frontend: Next.js 14, Tailwind CSS, Chart.js.*
- *Backend: FastAPI (Python), Pandas/Numpy (Data processing).*
- *AI Model: GLM 5.1 (via API).*
- *Deployment: Vercel (Frontend) & AWS Lambda/EC2 (Backend logic).*

© UMHackathon 2026

5. Key Data Flows

5.1 Data Flow Diagram (DFD)

Data moves from the User (External Entity) through the Refining Process. It is temporarily stored in a S3 Bucket (Data Store). The Audit Log Process captures every transformation step, ensuring no data is "lost" in the black hole of AI reasoning.

5.2 Database Schema (3NF)

We use a relational structure to track "Jobs" rather than storing the heavy raw data in the DB:

- ***Jobs Table: ID, UserID, Filename, Status (Pending/Success/Failed).***
- ***AuditTrail Table: JobID, StepDescription, CodeSnippetUsed, Timestamp.***

---

6. Functional Requirements & Scope (MVP)

#	Feature	Description
1	Autonomous Cleaning	AI writes Python code to handle missing values and unit conversions.
2	Self-Healing Loop	Automatic retry logic if the generated cleaning script crashes.
3	Industrial Dashboard	Visualizes cleaned O&G metrics (Pressure/Temp) immediately.
4	Audit Trail	Exportable PDF/Log showing exactly how the AI modified the data.

---

7. Non-Functional Requirements (NFRs)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Quality	Requirement	Implementation
Reliability	99% script success rate.	Stateful "retry" logic in the backend.
Token Cost	$Cost \leq 0.50$ per session.	Caching common O&G schemas (e.g., standard sensor headers).
Latency	Clean data in < 60s.	Asynchronous processing with a "Typewriter" progress UI.
Security	Data Isolation.	Every script runs in a fresh, temporary sandbox.

---

8. Monitoring & Evaluation

- Grayscale Rollout: Initially used for specific O&G datasets (e.g., just Drilling logs) before expanding to Refinery logs.
- Priority Matrix: * P1 (Critical): AI suggests a cleaning step that changes data values by >50% (triggers manual flag).
    - P2 (High): Script execution fails after 3 self-healing attempts.
- Emergency Rollback: If the system detects a logic flaw in the GLM output, it reverts to the raw "Safe" version of the file instantly.

---

9. Project Management & Timeline

- Day 1-3: Schema design and Prompt Engineering for GLM 5.1.
- Day 4-7: Backend Sandbox development & Frontend UI components.
- Day 8-10: QA Testing with real "messy" O&G log samples and Pitch prep.

Team Roles:

- Member A (Frontend/UI): Dashboard & Data Visualization.
- Member B (Lead Engineer): Backend Orchestration & GLM Integration.
- Member C (QA/Industrial Lead): Audit Trail logic & Domain validation (O&G standards).

© UMHackathon 2026

Email: umhackathon@um.edu.my